

8장 같은 피처를 두 번 만들지 않으려면

7주차 · 피처 스토어

1장은 데이터 엔지니어링과 MLOps가 만나는 자리가 피처(feature)이고 바로 그 자리에서 학습-서빙 불일치가 생긴다고 예고했다. 이 장은 그 예고를 회수한다. 7장 배치가 확정한 일별 집계를 모델이 소비할 수 있는 계약으로 바꾸는 창구, 즉 피처 스토어(feature store)를 최소 구성으로 세우고 두 경로가 정말 같은 값을 읽는지 실측으로 대조한다.

1. 오늘의 질문

- 훈련 코드와 서빙 코드가 각자 "전일 민원 건수"를 계산하면 무엇이 어긋나는가?
- 같은 피처를 왜 두 개의 저장소에 나누어 두는가, 그리고 두 곳의 값이 같다는 것을 무엇으로 보이는가?
- 7월 2일 오전 9시의 훈련 샘플에 그날 하루치 집계를 붙이면 왜 안 되는가?
- 이 피처를 누가 책임지고 그 값이 언제 알려진 것인지를 어디에 적어 두는가?

2. 표준 레시피가 있는 주방으로 보는 이 장

이 장의 네 개념은 본사 레시피로 운영되는 프랜차이즈 주방에 하나씩 대응한다.

프랜차이즈 주방	피처 스토어
본사가 배포한 레시피 문서 하나로 조리한다 — 지점마다 눈대중으로 만들지 않는다	피처 정의를 코드로 한 번 선언하고, 훈련과 서빙이 같은 정의를 읽는다
창고에 쌓아 둔 재료 전량 / 오늘 조리대에 꺼내 놓은 분량	offline store(이력 전체) / online store(키별 최신값)
아침마다 창고에서 조리대로 그날 쓸 분량을 옮긴다	materialize(오프라인에서 온라인으로 적재)
어제 낸 요리를 재현하면서 오늘 아침 들어온 재료를 쓰면 그것은 어제의 요리가 아니다	미래 누출과 시점 조인
유통기한이 지난 재료는 창고에 있어도 쓰지 않는다	TTL(Time-To-Live)
레시피 문서에 담당 부서와 재료 원산지를 적는다	피처 정의서의 소유자·원천·타임스탬프 의미

비유가 깨지는 지점을 한 줄로 밝힌다. 재료는 꺼내 쓰면 줄어들지만 피처 값은 여러 모델이 동시에 읽어도 줄지 않는다. 주방은 재고를 세면 관리가 되지만, 피처 스토어에서는 값이 줄지 않는 대신 **두 곳의 값이 정말 같은지를 따로 대조해야** 한다.

3. 같은 숫자를 두 곳에서 만들면 반드시 어긋난다

"지역구별 전일 민원 건수"를 입력으로 쓰는 민원 급증 예측 모델을 생각하자. 피쳐 스토어가 없으면 이 수치는 두 곳에서 따로 산출된다. 훈련 파이프라인은 과거 데이터 위에서 pandas로 집계하고, 서빙 API는 요청이 올 때마다 운영 DB에 SQL을 실행한다. 두 코드는 같은 것을 계산하려는 서로 다른 구현이고, 서로 다른 구현은 반드시 어긋난다.

7장의 7월 2일 실측이 이 어긋남을 그대로 보여 준다. 그날의 물리 레코드는 18건, 중복을 제거한 유효 건수는 17건, 표준 코드 매핑까지 마친 확정 집계는 16건이었다. 세 팀이 각자 "7/2 민원 건수" 피쳐를 만들면 훈련 파이프라인은 확정 집계를 써서 **16**, 서빙 API는 운영 DB를 그대로 세어 **18**, 분석 부서는 중복만 제거하고 미매핑을 포함해 **17**을 얻는다. 셋 다 "그날 민원 건수"라는 같은 이름을 달고 있지만 어느 것도 버그가 아니다. 정제 단계 중 어디까지를 적용했는가의 차이일 뿐이다.

문제는 모델이 16으로 훈련되고 18로 서빙될 때 생긴다. 입력 분포가 체계적으로 밀리고, 두 코드는 오류 없이 계속 돌기 때문에 성능 저하의 원인이 몇 주간 모델 탕으로 추적된다. 이것이 **훈련-서빙 스큐 (training-serving skew)**이며, 1장이 학습-서빙 불일치라고 부른 것과 같은 현상이다. 훈련이 곧 학습이고 서빙 시점에 값이 어긋난다는 실체가 같으며, 이 장은 피쳐 계산의 관점에서 스큐라고 부른다.

피쳐 스토어의 발상은 계산을 두 번 하지 않는 것이다. 한 번 정의하고, 한 곳에 저장하고, 훈련과 서빙이 같은 곳에서 조회한다. 구체적으로 세 가지가 일원화된다.

- **정의의 일원화:** 이 피쳐가 어떤 원천에서 어떤 스키마로 계산되는지를 코드로 선언하고 버전을 붙인다. 실습에서는 entity(조인 키)와 feature view(피쳐 묶음)의 선언이 여기에 해당한다.
- **저장의 일원화:** 값은 피쳐 스토어의 저장소에 있고, 훈련용 과거 이력과 서빙용 최신값이 이중으로 유지된다(4절).
- **조회 일원화:** 훈련은 `get_historical_features` 로, 서빙은 `get_online_features` 로 읽는다. API는 다르지만 같은 정의에서 나온 같은 값을 읽는다.

여기서 중요한 것은 피쳐 스토어가 16과 18 중 무엇이 옳은지를 판정하지 않는다는 점이다. 하나를 정의로 선택하게 하고 모두가 그것을 쓰게 강제할 뿐이다. 그리고 이 정의의 원천은 새 계산이 아니라 7장까지 만들어 온 확정 집계다. 피쳐 스토어는 배치 파이프라인이라는 생산자와 모델이라는 소비자 사이의 창구다.

반대 논거도 함께 확인한다. 모델이 하나뿐이고 배치 추론만 하며 피쳐 계산이 훈련 파이프라인 안에서 끝난다면, 저장소를 하나 더 들이는 것은 관리 대상만 늘린다. 훈련과 서빙이 같은 배치 코드에서 나오면 스큐 자체가 성립하지 않기 때문이다. 이중 구조가 필요해지는 조건은 대체로 셋 중 하나다. 온라인 서빙이 있어 소비 경로가 실제로 둘이거나, 여러 모델과 팀이 같은 피쳐를 재사용하거나, 미래 누출 감사가 요구되는 도메인이거나.

4. 저장소를 둘로 나눈다 — offline store와 online store

훈련과 서빙은 같은 피처를 전혀 다르게 소비한다. 훈련은 "지난 석 달분, 모든 지역구, 모든 날짜"를 한 번에 읽으므로 대량이고 과거 이력이 필요하며 몇 분쯤 걸려도 된다. 서빙은 "지금, 강남구, 최신값 하나"를 밀리초 안에 읽어야 하므로 소량이고 최신값만 있으면 되며 지연을 허용하지 않는다. 하나의 저장소로 두 패턴을 모두 잘하기는 어려워서 저장을 둘로 나눈다.

offline store는 피처의 전체 이력을 쌓는 곳이고 실습에서는 parquet 파일이 그 역할을 한다. 운영에서는 데이터 웨어하우스가 흔히 쓰인다. **online store**는 키별 최신값만 두는 곳으로 실습에서는 SQLite, 운영에서는 Redis 같은 키-값 저장소를 쓴다. 실습의 피처 원천은 7장 확정 집계 9행(3일 × 3개 지역구)이고, 이 9행이 모두 들어 있는 곳이 offline store다. **materialize**는 오프라인의 최신값을 온라인으로 옮기는 배치 작업이며, 이 작업을 거치면 online store에는 지역구당 최신 1행씩 3행만 남는다.

materialize 주기를 정하는 순서는 거꾸로다. 먼저 모델이 견딜 수 있는 피처의 낡음을 정하고, 적재 주기를 그 안쪽으로 잡는다. 낡음의 정도는 **피처 신선도**로 재는데, 온라인 스토어에 있는 값의 타임스탬프와 현재 시각의 차이다. 하루 한 번 적재하면 이 값의 최댓값은 약 24시간이 된다. 24시간이 괜찮은지는 두 가지가 정한다. 피처가 가리키는 현상이 얼마나 빨리 변하는가, 그리고 그 값으로 내린 결정을 되돌리는 비용이 얼마인가. "전일 민원 건수"로 다음 날 인력 배치를 정한다면 하루 지연은 정의상 문제가 아니지만, 재난 신고 밀집도로 출동 순서를 정한다면 24시간 전 값은 쓸 수 없고 잘못 정한 순서를 되돌릴 방법도 없다.

이 실습을 구성하면서 실제로 겪은 오류가 8장 버전의 침묵 실패다. 처음에는 적재를

`materialize_incremental` (증분 적재)로 호출했는데, 이 함수의 기본 시작점이 "현재 시각 빼기 TTL"이다. 실습 피처의 타임스탬프(7/2~7/4)는 그 창보다 과거라서 아무것도 적재되지 않았고, 에러도 나지 않았다. 온라인 조회가 조용히 `None` 을 돌려줄 뿐이었다. 4장의 유실, 6장의 폐기, 7장의 정제 drop에 이어 또 한 번 같은 교훈이 나온다. 비어 있는 결과는 스스로 알리지 않으므로 적재 작업의 성공 로그가 아니라 **적재된 건수**를 확인해야 한다. 만약 "값이 없으면 0으로" 같은 방어적 기본값이 코드에 있었다면 모든 지역구의 피처가 0인 채로 서빙됐을 것이다. 결측은 기본값으로 덮지 말고 **피처 결측률**로 계수한다(11장).

같은 값이 두 곳에 있으므로 두 곳이 정말 같은지가 곧 품질 항목이 된다. 실습은 세 지역구의 오프라인 최신값과 온라인 조회값을 기계적으로 대조해 **훈련-서빙 일관성 점검표**를 만들었다. 마포구(11440)는 6건, 관악구(11620)는 5건, 강남구(11680)는 9건이고 미매핑율은 세 곳 모두 0.045로, 오프라인과 온라인이 세 키 전부에서 같았다(`checked_keys` 는 3, `all_consistent` 는 true). 값 자체도 7장 7월 3일 실측과 같다. 배치 산출물이 변형 없이 피처가 되었다는 뜻이다.

이 점검표가 자명해 보인다면 그것이 요점이다. 자명함은 두 저장소를 같은 정의에서 적재했기 때문에 온 것이지 저절로 오지 않으며, 3절의 각자 계산 구조에서는 이 표의 어느 행이든 어긋날 수 있다. 다만 저장된 값이 같다고 안심할 수는 없다. 서빙 코드가 값을 읽은 뒤 다르게 변형하면 모델이 받는 입력은 여전히 어긋나므로, 운영에서는 서빙 시점에 모델로 들어간 값을 로그로 남겨 훈련 분포와 정기 비교한다(11장).

5. 그 시점에 알 수 있었던 값만 붙인다 — point-in-time correctness

이제 훈련 데이터를 만든다. 훈련 샘플 하나는 "7월 2일 오전 9시의 강남구" 같은 키와 시점의 쌍이고, 여기에 "전일 민원 건수" 피처를 붙여야 한다. 이때 7월 2일 하루 전체의 집계를 붙이면 어떻게 되는가. 그 값은 7월 2일 오전 9시에는 아직 존재하지 않았다. 그날이 끝나야 확정되기 때문이다. 미래의 정보가 과거 시점의 샘플에 스며든 것을 ****미래 누출(future leakage)****이라고 한다. 누출된 모델은 훈련과 평가에서 비현실적으로 좋게 나오다가 실제 서빙에서 성능이 급격히 떨어진다. 성능이 좋아 보여서 더 위험한 결함이다.

point-in-time correctness는 이것을 조회 규칙으로 막는다. 각 샘플의 기준 시점마다 **그 시점 이전에 알려진 가장 최근 값만** 조인한다. 그래서 피처의 타임스탬프가 결정적으로 중요해지는데, 그 타임스탬프는 "값이 가리키는 기간"이 아니라 "값이 알려진 시점"이어야 한다. 실습의 설계 결정이 정확히 이것이다. 7월 1일의 일별 집계는 그날이 끝나야 확정되므로 타임스탬프를 7/1이 아니라 **7/2 00:00**으로 부여했다. 2장의 이벤트 시간과 처리 시간, 6장의 워터마크, 7장의 logical date에 이어 네 번째로 확인하는 시간 구분이며, 이번에는 발생한 시각도 처리한 시각도 아닌 "알 수 있게 된 시각"이다.

실습은 기준 시점이 다른 훈련 샘플 4개로 시점 조인을 실행했다.

훈련 샘플(키, 기준 시점)	그 시점까지 확정된 집계	조인된 건수(실측)	검산(7장 실측)
관악구, 7/1 09:00	없음(첫 확정은 7/2 00:00)	행 자체가 탈락	—
강남구, 7/2 09:00	7/1까지	8 (미매핑율 0.05)	7/1 강남 8
마포구, 7/3 09:00	7/2까지	5 (0.059)	7/2 마포 5
강남구, 7/4 09:00	7/3까지	9 (0.045)	7/3 강남 9

둘째 행을 규칙 그대로 손으로 따라가 보자. 강남구의 피처 값은 타임스탬프 7/2 00:00, 7/3 00:00, 7/4 00:00에 존재한다. 기준 시점 7/2 09:00 이전인 것은 7/2 00:00 하나뿐이므로 7/3과 7/4의 값은 미래라서 후보에서 빠진다. 남은 후보 중 가장 최근인 7/2 00:00의 값 8이 붙고, 기준 시점과의 간격 9시간은 TTL 3일 안이라 유효하다. 엔진의 실측 출력 8이 손 검산과 정확히 일치한다.

여기서 강남구 샘플 두 개에 다른 값 8과 9가 붙었다는 점을 확인해야 한다. 같은 키라도 기준 시점이 다르면 그때 알 수 있었던 값이 다르기 때문이고, 이것이 시점 조인의 정의다. 7/2 09:00 샘플에 그날 완성될 집계가 아니라 7/1의 8이 붙은 것이 바로 미래 누출 방지의 직접 증거다. 만약 두 값이 같게 나온다면 시점 조인이 아니라 최신값 조인을 하고 있는 것이고, 그것이 곧 누출이다.

표의 첫 행은 예상과 다른 실측이다. 7/1 09:00의 관악구 샘플은 조인할 피처가 없는 시점인데, 결과에서 피처가 빈 값으로 남는 것이 아니라 **행 전체가 사라졌다**. 입력 4행에 결과 3행이고, 탈락 행은 증거 파일에만 기록된다. 어떤 오류도 없이 훈련 셋이 줄어드는 조용한 샘플 손실이며, 특히 이력이 짧은 신규 키의 샘플이 체계적으로 빠지므로 무작위 손실이 아니라 **편향된 손실**이다. 대응은 7장 정제의 총계 보존과 같은 구조다. 조인 전후의 행 수를 반드시 대조하고, 차이가 나면 어느 샘플이 왜 빠졌는지 확인한다.

시점 조인에는 하한도 있다. 실습의 feature view는 TTL 3일을 선언했고, 기준 시점보다 3일 넘게 오래된 값은 그 시점에 알려져 있었더라도 붙이지 않는다. 석 달 전의 "전일 민원 건수"는 알려진 값이긴 해도 "전일"이라는 피처의 의미를 잃기 때문이다. 6장의 워터마크가 "얼마나 늦은 데이터까지 받을 것인가"라면 TTL은 "얼마나 오래된 피처까지 유효로 볼 것인가"이며, 둘 다 선언 하나로 시간의 경계를 정하고 경계 밖을 제외한다. 그래서 7장 배치 실패가 며칠 이어져 피처 공급이 끊기면, TTL이 지난 시점부터 훈련 샘플이 조용히 사라지기 시작한다.

기준 시점을 어디로 잡을지도 결과를 바꾼다. 실습은 각 날짜의 09:00을 썼는데, 임의의 값이 아니라 "오전 9시에 그날의 인력 배치를 정한다"는 업무 시나리오에서 나왔다. 결정은 09:00에 내리는데 훈련 샘플의 기준 시점을 23:59로 두면 그날 낮에 확정된 값이 붙어 설정값 하나로 미래 누출이 들어온다. 반대로 자정 기준으로 훈련하고 09:00에 서빙하면 누출은 없지만 서빙 쪽이 더 신선한 값을 받아 스쿼의 시간 버전이 된다. 규칙은 하나다. **훈련 샘플의 기준 시점은 운영에서 실제로 예측이 호출되는 시각과 같아야 한다.**

6. 피처 정의서 — 누가 책임지고 언제 알려진 값인가

피처가 한 곳에 정의되고 여러 모델이 재사용하는 자산이 되는 순간, 기술 문제와 함께 책임 문제가 온다. 원천 부서가 집계 방식을 바꿨는데 소비하는 모델 세 개가 그것을 모르면 사고가 난다. 그래서 피처 정의서에는 계산 명세만이 아니라 소유와 책임이 들어간다.

- **소유자**: 이 피처의 정의와 품질에 책임지는 부서다. 원천이 바뀌면 소비자에게 알릴 의무가 여기에 붙는다.
- **원천과 갱신 주기**: 어느 시스템의 어떤 산출물에서 얼마나 자주 만들어지는가.
- **개인정보 등급**: 피처 단위로 표기한다. 지역 단위 집계는 비식별이지만 가구·개인 단위 피처는 접근 통제와 이용 목적 제한이 붙는다. 등급이 피처 단위로 붙는다는 점이 중요하다. "피처 스토어에 접근 가능"이라는 시스템 단위 권한은 개인정보 피처 앞에서 무의미하다.
- **타임스탬프 의미**: 5절의 "알려진 시점" 규약을 문서에 적는다. 미래 누출 감사에 답할 때 근거가 되는 문장이다.

실습은 이 항목들을 feature view의 태그로 선언하고 registry에서 추출해 `ch8_feature_definitions.json` 으로 내보냈다. 프로젝트는 `public_complaints`, entity는 `district`(조인 키 `lawd_cd`), feature view는 `complaint_daily`이고 여기에 `complaint_count`(Int64, 전일 확정 민원 건수)와 `day_unmapped_rate`(Float64, 전일 매핑 실패율) 두 피처가 담긴다. TTL은 3일, 소유자는 민원데이터팀(가상), 원천은 7장 일별 배치 확정 산출물, 갱신은 일 1회, 개인정보는 없음이며, 타임스탬프의 의미는 "값이 확정·공개된 시점, 즉 집계 대상일 다음 날 00:00 UTC"로 명시되어 있다.

조인 키가 지역명이 아니라 법정동 시군구 코드라는 점을 눈여겨보자. 7장에서 "이름이 아니라 코드로 붙인다"고 했던 표준 코드 원칙이 피처 스토어의 entity 설계에서 그대로 반복된다. entity는 형식상 조인 키의 선언이지만 실제로 고르는 것은 피처가 서술하는 대상의 단위이고, 무엇이 옳은지는 **서빙 시점에 무엇을 키로 조회하는가**가 정한다. 상황실이 "강남구의 현재 상태"를 물으면 키는 지역구여야 하고, 접수

창구가 "이 민원 건의 예상 처리 시간"을 물으면 키는 민원 건 ID여야 한다. 단위를 잘게 잡으면 피처는 정밀해지지만 온라인 스토어의 키 개수가 늘어 적재 비용이 커지고, 키마다 이력이 짧아져 5절의 행 탈락이 늘어난다.

정의는 불변이 아니다. 정제 규칙이 바뀌면 같은 이름의 피처가 다른 값을 갖게 되는데, 기존 정의를 조용히 덮어쓰면 그 피처로 훈련된 모델들의 훈련-서빙 일치가 소급으로 무효가 된다. 스큐를 막으려던 구조가 스큐의 원인이 되는 역설이다. 그래서 정의 변경은 새 feature service 버전으로 내고 기존 버전은 소비 모델이 이전을 마칠 때까지 유지한다. 실습 service 이름 `complaint_model_v1`의 v1은 장식이 아니라 이 규율의 자리 표시자다.

이 정의서는 8장의 종착점이 아니라 다음 장들의 입력이다. 9장은 이 피처로 만든 학습 데이터의 지문 해시 (`data_fingerprint_sha256`)를 실험 기록에 남기고, 14장 검수표는 통합 시스템이 실제로 처리한 데이터의 지문을 그 값과 대조한다. 두 값이 일치해야 "9장에서 학습에 쓴 데이터"와 "운영이 처리한 데이터"가 같다고 말할 수 있다. 정의서가 무엇을 계산했는지를 문장으로 남긴다면, 지문 해시는 그 계산의 결과가 바뀌지 않았음을 값 하나로 남긴다.

7. 실습 8.1 Feast 기반 최소 피처 저장소 (난이도 ★★★★★)

실습 목표

entity-feature view-feature service를 등록하고 시점 조인·materialize·온라인 조회를 차례로 실행한다. 시점 조인의 행 탈락과 온라인 스토어의 행 수, 그리고 오프라인과 온라인의 값 일치를 실행 산출물로 확인한다.

실행 환경

Python 3.10 이상(실습 검증은 3.13), 외부 인프라는 필요 없다. offline store는 parquet 파일, online store는 SQLite, registry는 로컬 파일을 쓴다.

실행 명령

```
cd practice/chapter8
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter8.py
```

핵심 코드

두 소비 경로의 API가 다르다는 것이 이중 구조의 요약이다.

```

train_df = store.get_historical_features(entity_df, features=svc).to_df() # 훈련: 시점 조인
store.materialize(start_date=t0, end_date=now) # 적재(범위 명시)
serving = store.get_online_features(features=svc,
                                    entity_rows=[{"lawd_cd": "11680"}]) # 서빙: 최신값

```

전체 코드는 [practice/chapter8/code/8-1-feature-store-minimal.py](#) 참고

실행 전에 적는 예측

실행하기 전에 세 가지를 종이에 적고 각 칸에 근거를 한 줄씩 붙인다. 근거 없는 숫자는 대조해도 배울 것이 없다.

1. 입력 4행으로 시점 조인을 하면 결과는 몇 행인가.
2. 원천 9행을 materialize하면 온라인 스토어는 몇 행이 되는가.
3. 강남구의 두 샘플(7/2 09:00, 7/4 09:00)에는 각각 어떤 값이 붙는가.

실행 결과와 대조

항목	실측	확인할 것
시점 조인 결과 행 수 (입력 4행)	3 (탈락: 관악구 7/1 09:00)	시점 조인은 입력 행 수를 보존하지 않는다
온라인 스토어 행 수(원천 9행)	3 (지역구당 최신 1행)	materialize는 이력 복사가 아니라 최신값만 남기는 작업이다
강남구 두 샘플에 붙은 값	8(7/2 09:00) / 9(7/4 09:00)	같은 키라도 기준 시점이 다르면 값이 다르다
온라인 조회와 오프라인 최신값	마포 6 · 관악 5 · 강남 9(미매핑율 0.045)	all_consistent 가 true, checked_keys 가 3

조인 행 수를 4로 적었다면 유효 피처가 없는 샘플이 빈 값으로 남지 않고 사라진다는 것을 놓친 것이다. 손실이 무작위가 아니라 이력이 짧은 키에 몰린다는 점이 문제다. 온라인 행 수를 9로 적었다면 두 저장소의 성격 차이를 놓친 것이고, 실제로 9행이 나왔다면 오히려 설정이 잘못된 것이다. 강남구 두 값을 같게 적었다면 지금 하고 있는 것은 시점 조인이 아니라 최신값 조인이다.

TTL도 검산해 보자. 실습의 요구사항은 "배치가 하루 실패한 적은 있고 이를 연속 실패한 적은 없다"였다. TTL을 1일로 잡았다면 배치가 하루 실패한 다음 날부터 유효한 값이 없어 훈련 샘플이 통째로 빠진다. 3일은 이를 연속 실패까지 견디고 하루의 여유를 남긴 값이다. 길게 잡으면 안전하지만 오래된 값이 "전일 건수"라는 이름을 달고 붙는 신선도 손실을 받는다. TTL은 디스크나 성능이 아니라 배치 실패를 며칠까지 견딜 것인가에서 역산한다.

산출물은 세 개다. `ch8_feature_definitions.json` 은 registry에서 추출한 피쳐 정의서, `ch8_consistency_report.json` 은 훈련-서빙 일관성 점검표, `ch8_feature_report.json` 은 시점 조인 결과와 탈락 행과 일관성 점검을 함께 담은 이 장의 증거 파일이다. 마지막 줄에 `CH8_RUN_PASS` 가 찍히면 정상 종료다.

8. 핵심 요약

- 1. 피쳐를 두 곳에서 따로 계산하면 반드시 어긋난다.** 7월 2일 민원 건수는 어디까지 정제했느냐에 따라 16, 17, 18로 갈리고 셋 다 버그가 아니다. 피쳐 스토어는 무엇이 옳은지 판정하지 않고 하나를 정의로 고르게 한 뒤 훈련과 서빙이 그것만 읽게 만든다. 1장이 예고한 학습-서빙 불일치를 구조로 막는 자리가 여기다.
- 2. 소비 패턴이 달라서 저장소를 둘로 나눈다.** 훈련은 이력 전체를 대량으로 읽고 서빙은 키 하나의 최신값을 밀리초에 읽으므로, 이력 9행은 offline store에 두고 materialize로 옮긴 최신 3행을 online store에 둔다. 적재는 성공 로그가 아니라 적재된 건수로 확인한다. 아무것도 적재되지 않아도 에러는 나지 않기 때문이다.
- 3. 두 저장소의 값이 같다는 것은 대조해서 보이는 것이다.** 실습은 세 지역구에서 오프라인 최신값과 온라인 조회값을 기계적으로 비교해 `all_consistent` 를 true로 남겼다. 이 자명함은 같은 정의에서 적재했기 때문에 온 것이며, 각자 계산 구조에서는 어느 행이든 어긋날 수 있다.
- 4. 훈련 데이터에는 그 시점에 알 수 있었던 값만 붙인다.** 피쳐 타임스탬프를 "값이 알려진 시점"으로 정의하고 기준 시점 이전의 가장 최근 값만 조인하면, 강남구의 두 샘플에 8과 9라는 서로 다른 값이 붙는다. 두 값이 같아지는 순간 그것은 미래 누출이다. 조인할 값이 없는 샘플은 조용히 행째 사라지므로 조인 전후 행 수 대조는 선택이 아니다.
- 5. 정의서는 피쳐가 자산이 되는 순간 필요해진다.** 소유자·원천·갱신 주기·개인정보 등급·타임스탬프 의미를 registry에서 뽑아 문서로 남기고, 정의가 바뀌면 덮어쓰지 않고 새 버전으로 낸다. 이 문서와 9장의 지문 해시가 함께 있어야 "무엇으로 학습했는가"에 답할 수 있다.

9. 과제

실습 8.1을 실행해 생성된 `practice/chapter8/data/output/ch8_feature_report.json` 을 LMS에 그대로 업로드한다. 이어서 다음 네 가지를 각각 세 문장 이내로 쓴다.

- 본인 파일의 `consistency` 블록에서 `all_consistent`, `checked_keys`, 그리고 키 하나의 `offline_latest` 와 `online` 값을 옮겨 적는다.
- 7장의 일별 배치가 **나흘 연속 실패**했다고 하자. 나흘째에 온라인 스토어에서 피쳐를 조회하면 무엇이 반환되는가. 본인의 `online` 값 구조와 TTL 3일을 근거로 답한다.
- 같은 시점에 오프라인 훈련 셋을 만들면 어떤 문제가 생기는가. 학습-서빙 불일치와 연결해 서술한다.
- 이 사고를 배치 실패 직후에 잡으려면 어느 지점에 무엇을 두어야 하는가. 감지 지점 두 개를 제시하고 각각이 무엇을 신호로 삼는지 쓴다.

수업일 포함 7일 이내에 제출한다.

10. 더 알아보기

이 장에서 다루지 않은 내용은 실무교재 `docs/chapter8.md`에 있다.

- **피처 접근 등급 설계와 개인정보 포함 피처의 이용 제한** — 공개 집계·내부 업무·개인정보의 3등급 설계표: `docs/chapter8.md` §8.4 (13장에서 집행으로 확장한다)
- **feature service를 무엇으로 묶을 것인가** — 모델 단위와 주제 단위의 비교, 판단 기준값이 되는 피처 재사용률: §8.4
- **민간 실시간 피처 공급의 공공 적용** — 추천·사기 탐지의 실시간 피처가 민원 배정·재난 긴급도 피처로 옮겨 가는 경로: §8.2
- **실습의 설계 단계와 위험 식별 표** — entity·집계 창·TTL·주기·묶음을 요구사항에서 스스로 도출하는 워크시트: `docs/chapter8.md` 실습 8.1의 1~4단계